

Introduction.....	1
GitHub Repository.....	2
Hardware Source Files.....	2
Firmware Source Files.....	4
Design Tasks.....	6
Task 1: Correlation Engine Parallelization.....	6
Task 2: AXI4 Lite Wrapper.....	13
Task 3: OMP Outer Loop (Bare-metal C).....	16
Task 4: End-to-End Heartbeat Reconstruction Pipeline.....	18
Post-Lab Discussion.....	21

Introduction

Wearable cardiac monitors must transmit data wirelessly through tissue. Implantable neural recorders must survive for years on just a coin-cell battery. A cochlear implant must decode complex speech in milliseconds without generating harmful levels of heat to surrounding tissue. The data coming out of these sensors has the possibility of saving someone's life, but the hardware they are built on is oftentimes severely constrained in terms of power, storage, and bandwidth.

Counterintuitively, Compressed Sensing theory proves that exact or near-exact signal recovery is possible even while cutting the number of measurements in half. The one condition is that the measurement vectors must be broadly spread across the sparse basis. Biomedical signals like the ones described above happen to satisfy this condition perfectly. For example, an electrocardiogram (ECG) is sparse in a wavelet basis because the QRS complex (the most prominent spike on an ECG) is localized transient.

This project implements a complete compressed sensing recovery pipeline for ECG signals on a Zynq z7-20 FPGA. I provide the structure and documentation needed to recreate my custom hardware accelerator that improves the computationally intensive search step of the Orthogonal Matching Pursuit (OMP) recovery algorithm by ***~18x*** (when compared to tuned Cortex-A9 software). The full reconstruction loop runs in sync across the programmable logic and ARM Cortex-A9 processor, and demonstrates clinically relevant cardiac signal reconstruction (PRD < 9%).

To validate the pipeline and to visually portray the accuracy of the reconstruction, recovered ECG windows are streamed from the processor to a host machine over a direct User Datagram Protocol (UDP) link. The host machine runs a receiving process with an active Python client, and visualizes the incoming data as an animation. For each reconstructed window, we print the current recovered signal by laying it on top of the original 256-sample ECG signal. The per-window PRD is portrayed in the title, and it is all updated once per 32 OMP iterations. This visualization portrays the quality of the reconstructed signal, and proves that hardware accelerated compressed sensing can produce clinically accurate cardiac signal reconstruction in real time.

This document contains everything you need to follow along in my design process for the Final Project, and is split up into four major sections based on the main design tasks. First the actual hardware accelerated component, the correlation engine, is described and documented. This was the heavy design task of the lab. Next I wrapped communication in AXI4 Lite protocol in order to drive the fabric through the Cortex-A9 processing side of the FPGA. Then I describe the outer loop of the recovery engine by walking you through the C side of the algorithm. Lastly, the UDP protocol and Python animation function is briefly explained, and the visualization pipeline is completed and demonstrated.

Four sub-sections live within the PL fabric design task 1; *Component Description* describes the current component under design, *Paper Designs* show the RTL level component workings mapped out, *Vivado Schematics* show the Vivado generated design which was compared to my paper design, *Simulations* showed annotated waveforms that provided evidence that the component was working as intended. The OMP outer loop C components, and the End-to-End pipeline are completed with basic logic and connections, so they don't have paper designs or elaborated diagrams. The Design Tasks are then followed by the post-lab discussion, which provides an insight into the lab workflow and outcomes. They were validated through repeated usage of the pipeline. Enjoy!

GitHub Repository

Hardware Source Files

RTL — [omp_accelerator/omp_accelerator.srcs/sources_1/new/](#)

File	Description	Subcomponents Instantiated
omp_params.vhd	System-wide constants package (N=256, M=128, MAC_BITS, accumulator width)	—

File	Description	Subcomponents Instantiated
mac_lane.vhd	Single MAC unit; truncate-before-multiply via arithmetic right shift	—
adder_tree32.vhd	Pipelined 32-input adder tree; reduces one chunk of MAC outputs	—
chunk_acc.vhd	Accumulates one 32-element chunk into the 38-bit (Q8.30) running sum	mac_lane ×32, adder_tree32
corr_datapath.vhd	Full inner product for one atom; 4 sequential chunk passes over M=128	chunk_acc
argmax_tracker.vhd	Running argmax with strict > tie-break (lowest index wins); guards abs() of most-negative value	—
dict_mem.vhd	Dictionary BRAM (128×256 Q1.15, atom-major, 2 values/word)	banked_mem
resid_mem.vhd	Residual BRAM (128 Q1.15, rewritten each OMP iteration)	banked_mem
banked_mem.vhd	Underlying banked BRAM primitive used by dict and residual memories	—
omp_mem_top.vhd	Memory subsystem; unified AXI-write / datapath-read interface with address decode	dict_mem, resid_mem
omp_engine.vhd	Engine top level; FSM (IDLE→COMPUTE→FINISH), iterates datapath over all 256 atoms	corr_datapath, omp_mem_top, argmax_tracker
axi_omp_corr.vhd	AXI4-Lite slave wrapper; exposes CTRL/STATUS/RESULT_IDX/BRAM_D/BRAM_R register map to PS	omp_engine

Testbenches — omp_accelerator/omp_accelerator.srscs/sim_1/new/

File	Description
tb_omp_engine.vhd	Validates all 5 golden vectors through the full engine + FSM
tb_corr_datapath.vhd	Unit test for the correlation datapath
tb_omp_mem_top.vhd	Verifies BRAM address decode for BRAM_D and BRAM_R regions

Firmware Source Files

Main Application — vitis_workspace_omp_accel/omp_accel_application/src/

File	Source	Description	Hardware Driven
omp_loop.c	Custom	Full OMP outer loop on Cortex-A9: writes residual to hardware, pulses START, polls DONE, reads RESULT_IDX, least-squares solve, residual update, inverse wavelet transform, UDP stream to host	AXI OMP Correlator, lwIP UDP
omp_engine_test.c	Custom	Hardware bring-up validation: loads each of 5 golden vectors over AXI, checks RESULT_IDX matches expected index, prints pass/fail over UART	AXI OMP Correlator
ecg_window.h	Generated (gen_window_header.py)	Embeds one ECG window: y_q15[], s_orig[], D_q15[], psi[], Y_SCALE_ALPHA	—
platform.c/h,	Xilinx lwIP 2.11	PS cache init, interrupt	Zynq PS7, GEM

File	Source	Description	Hardware Driven
platform_zynq.c, platform_config.h	example (Vitis 2021.2)	controller setup, GEM/lwIP netif initialization	
lscript.ld	Vitis 2021.2 (generated)	Linker script; defines DDR memory map for bare-metal application	—
The ECG Dataset	The MIT-BIH ECG Dataset	Not an actual file, but the dataset that we obtained heartbeat samples from	-

Patched BSP Files — software/bsp/

File	Source	Description	Hardware Driven
xemacpsif_physpeed.c	Xilinx lwIP 2.11 BSP, patched	Strips gigabit advertisement (ADVERTISE_1000) so Realtek RTL8211E and Zynq GEM agree at 100 Mbps full-duplex. Without this patch UDP never arrives at the host.	Zynq GEM, RTL8211E PHY
xparameters.h	Vitis 2021.2 (generated from XSA)	AXI base address map for this block design; kept for reference	—

Python Host Scripts — Final_Project/

File	Description
animate.py	Receives UDP frames from board (port 5005), animates OMP convergence iteration-by-iteration using matplotlib.FuncAnimation
omp_reference.py	Software-only OMP golden reference; hardware-exact argmax (truncate-before-multiply, strict > tie-break) used for PRD validation
data_collection/prepare_data.py	Pulls MIT-BIH ECG records via wfdb, builds wavelet dictionary $D=\Phi\psi$ (db4), saves per-window .npz to

File	Description
	signals/
data_collection/gen_window_header.py	Converts a signals/*.npz to ecg_window.h; applies y overflow pre-scale α
goldenVectors/make_golden.py	Generates the 5 golden test vectors (easy-margin, tight-margin, near index 0, near index 255, engineered tie)
goldenVectors/npz_to_c.py	Converts golden vector .npz files to C headers for Vitis firmware
goldenVectors/npz_to_tb.py	Converts golden vector .npz files to .mem format for VHDL testbench \$readmemh

Design Tasks

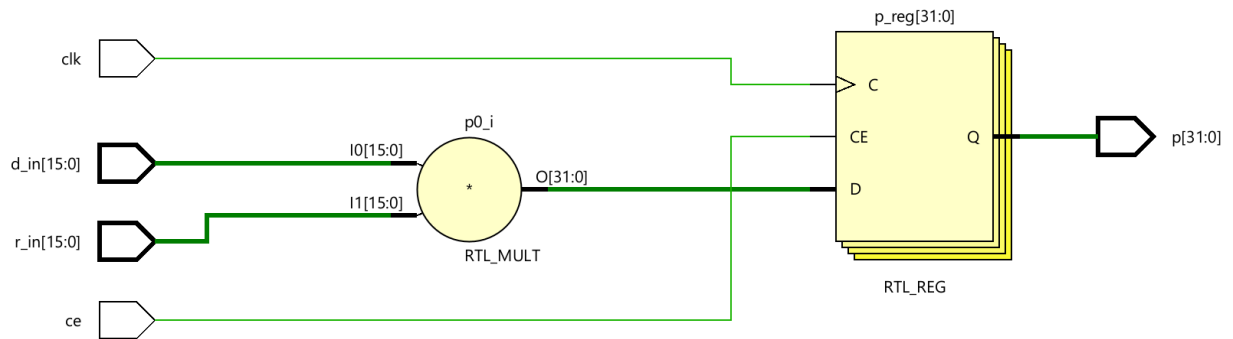
Task 1: Correlation Engine Parallelization

MAC Component

The Multiply-Accumulate (MAC) unit is the core number crunching unit of the correlation engine. Each MAC unit takes in one Q1.15 dictionary atom entry and one Q1.15 residual entry and multiplies them together into a 38-bit Q8.30 accumulator. The reason for 38-bits is to prevent overflows, as we are multiplying all of the 128 dictionary and residual terms into the accumulator. This is the first hard requirement for generics in this document.

Vivado Schematics

The humble MAC unit. As small as it may seem, this will accelerate the typical OMP correlation step by ***~18x*** by utilizing the parallel nature of the FPGA. It takes in the two Q1.15 vectors, and multiplies them. That's it. You will see below how they are all connected, and how powerful it becomes. The design for the MAC unit is trivial, so I chose not to perform a testbench writeup or a paper design.

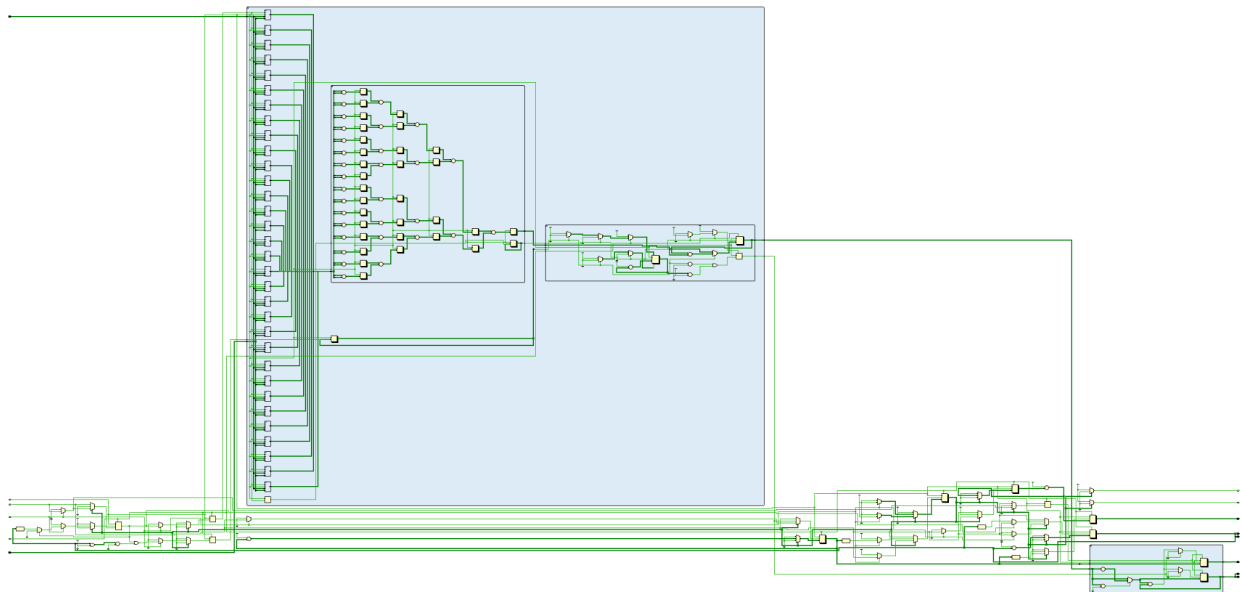


Parallel MAC and Adder Tree Component

Our ECG sample is 256-samples, but we are sensing with a compression ratio of 0.50 , so we are running this recovery algorithm across 128-samples. That means our hardware has to compute 128 16-bit by 16-bit multiplications. We do this by tapping into the parallel nature of the FPGA, and instantiate 32 parallel MAC units to process this step in 4 chunks (128 samples / 32 MACs), accumulating the result in the 38-bit accumulator. In order to add together 128 multiply outputs, an adder tree was implemented to sum down the output through 5 pipeline stages ($\log_2(32)$). This parallelism is the core reason for the **$\sim 18x$** speedup.

Vivado Schematics

Below is the RTL elaborated design for the Parallel MAC and Adder Tree. You can see the line of MAC blocks on the left, this is the heavy computational engine behind the OMP reconstruction loop. Then the results cascade down through the Adder Tree, summing the full 32 multiplications in 5 cycles. I did not make a paper design for this component, but I should have, I just ran out of time. The design would have been trivial, as even though the design below looks complex, it really is just routing signals between 32 separate MAC arrays and then the adder tree collapsing down to a single value.



Vivado RTL Schematic Elaborated Design for Parallel MAC and Adder Tree

Argmax Tracker Component

Over the 128 multiplies, we need to track the largest one, as that is the vector that we will use to update our residual dictionary. The argmax tracker does just that by implementing a strict less-than comparison, as opposed to a less-than or equal-to. This is a decision that theory decides (the lowest index wins in a tied score), and it is implemented the same as the software baseline counterpart. There is one edge case to guard against. Since these 38-bit numbers are two's complement, the MSB is actually our sign bit, and if it overflows, can represent the wrong signed number. This has to be explicitly checked to guard against a false max score.

Paper Designs

Below is the paper design for the *argmax_tracker*. Notice that we also have to keep track of the index of the largest term, in order to correctly update the residual dictionary with the right index.

RTL Schematic : ArgMax

argmax_tracker

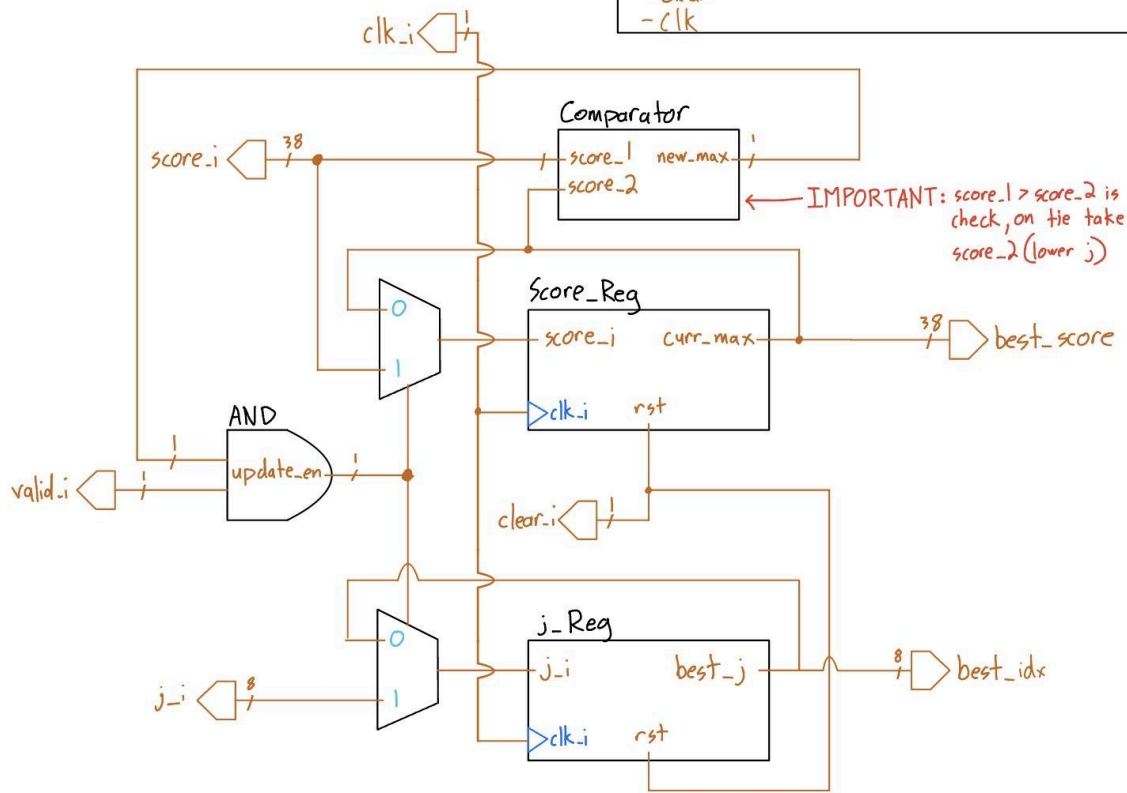
Tracks max score per itr.

Inputs:

- score
- j
- valid
- clear
- clk

Outputs:

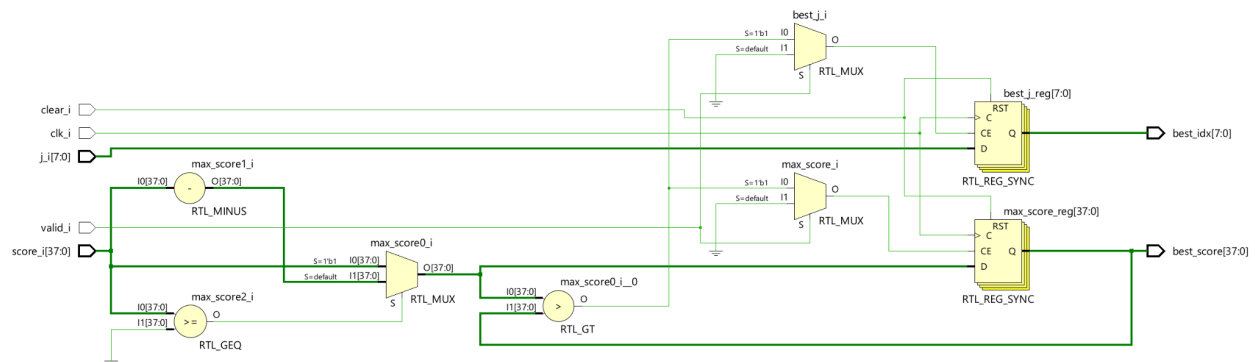
- best_idx
- best_val



RTL Schematic Paper Design for the *argmax_tracker*

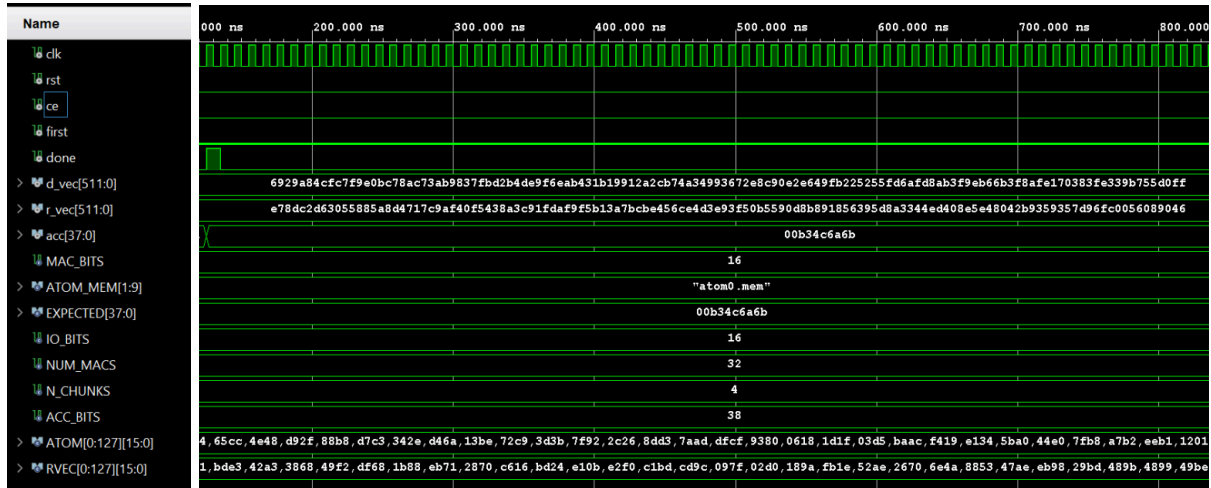
Vivado Schematics

Below is the Vivado generated elaborated design. It closely resembles my paper design with the registers and the logical blocks routing signals.

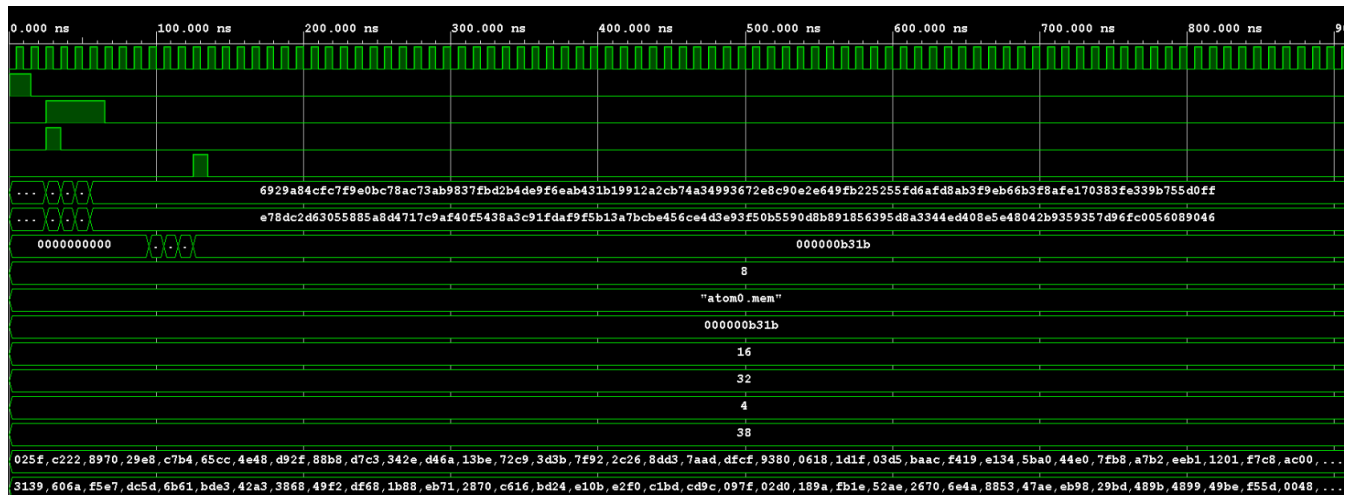


Simulations

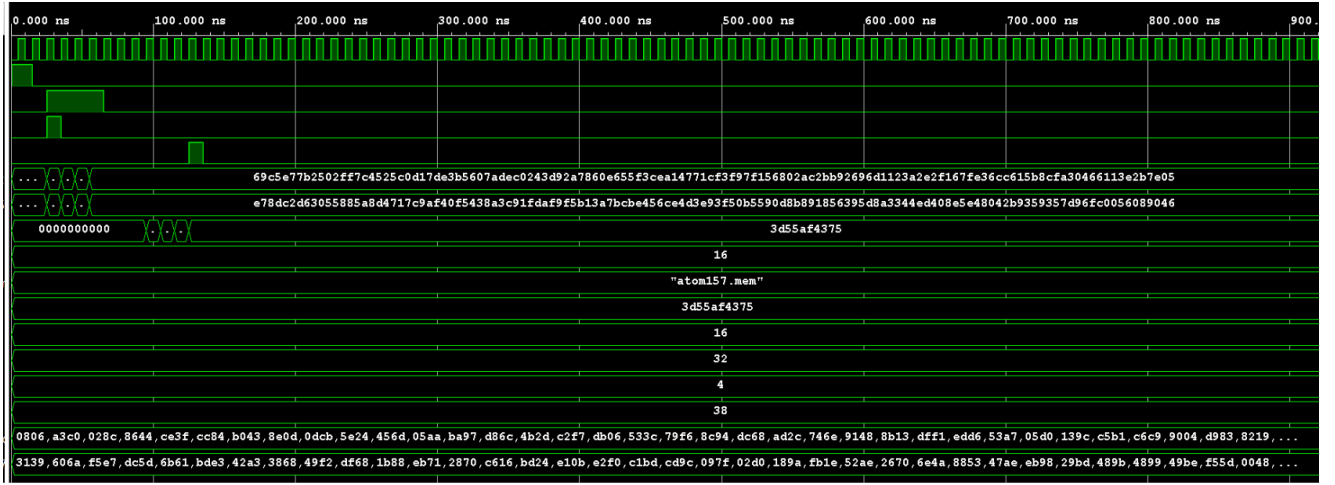
Below is a suite of tests that validate the MAC wrapper works as intended. I give the system 2 512-bit numbers (essentially I provide the system the data needed to perform one OMP iteration), I ask it to multiply them together, and I compare the hardware result against the expected result obtained from software (more on this below). I ran 4 tests, each with a different input, testing various numbers and edge cases. In the test cases, you can see the *acc* signal, which is the output of the accumulator after the adder tree. Then you can see the *EXPECTED* signal, showing what the output of the *acc* should be (via software). This matches up with every test case below.



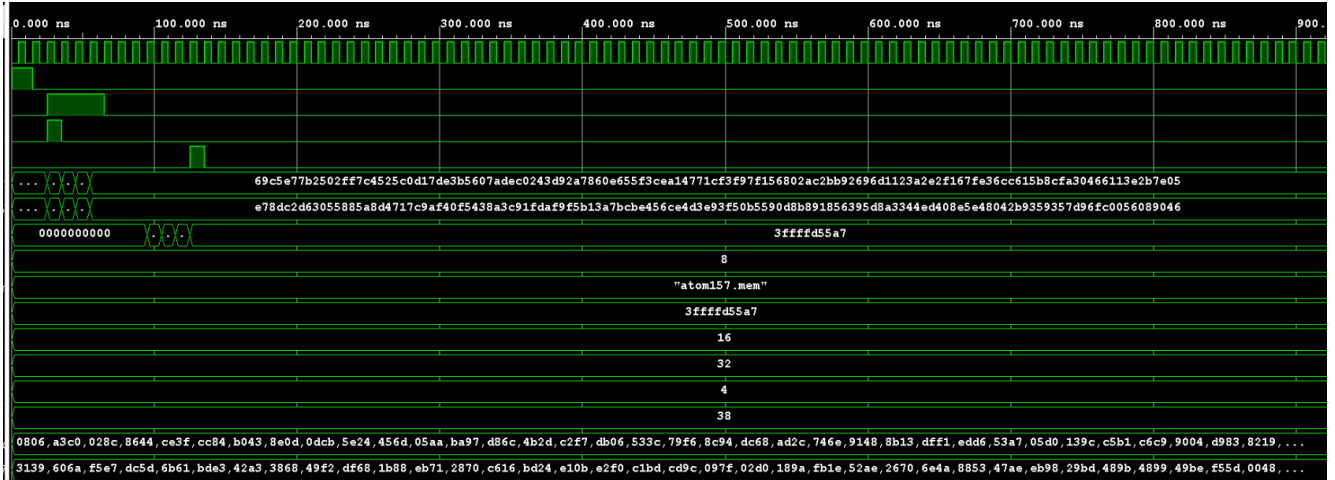
OMP Correlation Datapath Simulation showing correct accumulator values Test 1



OMP Correlation Datapath Simulation showing correct accumulator values Test 2



OMP Correlation Datapath Simulation showing correct accumulator values Test 3



OMP Correlation Datapath Simulation showing correct accumulator values Test 4

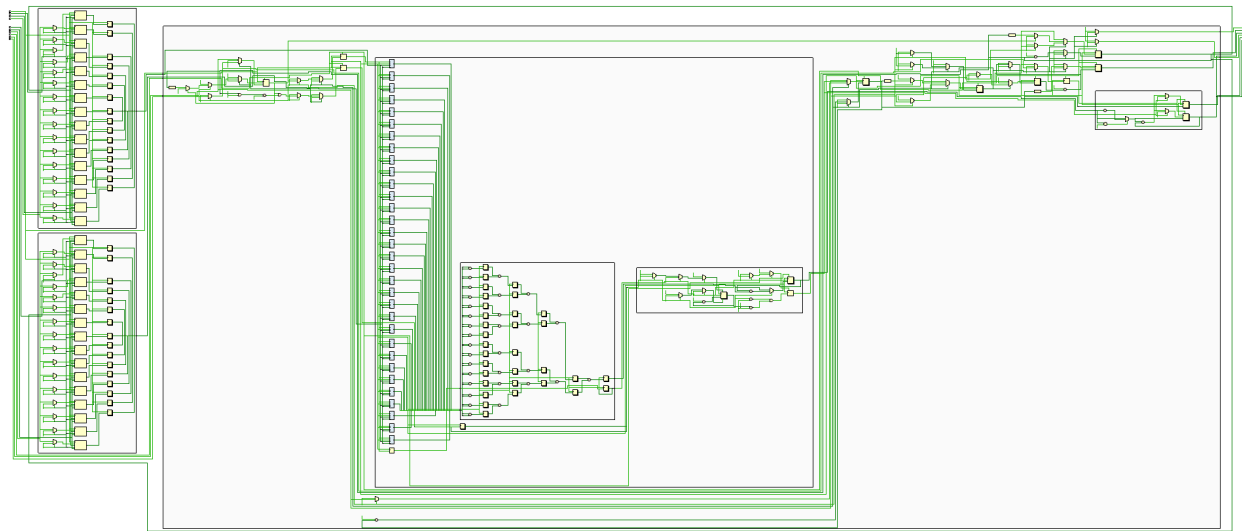
Top Level FSM Component

The correlation engine is controlled by a three state Finite State Machine with states *Idle*, *Compute*, and *Finish*. In *Idle*, the engine waits for the AXI control register to obtain the START pulse generated by the PS. Once received, the *Compute* is active, where it iterates through all 256 atoms generating the 128 inner product multiplies. The *argmax_tracker* keeps track of the maximum support, and after the last one is processed, we move into the *Finish* state. This is where we assert DONE to our AXI control register, and keep the *best_score* and *best_idx* for the software to grab. The processor is responsible for writing the new residual dictionary to the

BRAM_R before each START pulse, and it enforces the OMP iteration cap by only calling this FSM 32 times.

Vivado Schematics

Here is the Vivado generated elaborated design. You can see the memory banks on the left (the D and r BRAMs) that feed into the 32 parallelized MAC units. Then near the middle you can see the adder tree collapsing the 32 outputs down to a single 38-bit number. Then the logic on the right which pertains to the *argmax_tracker* and other glue logic needed to interface with the external world.



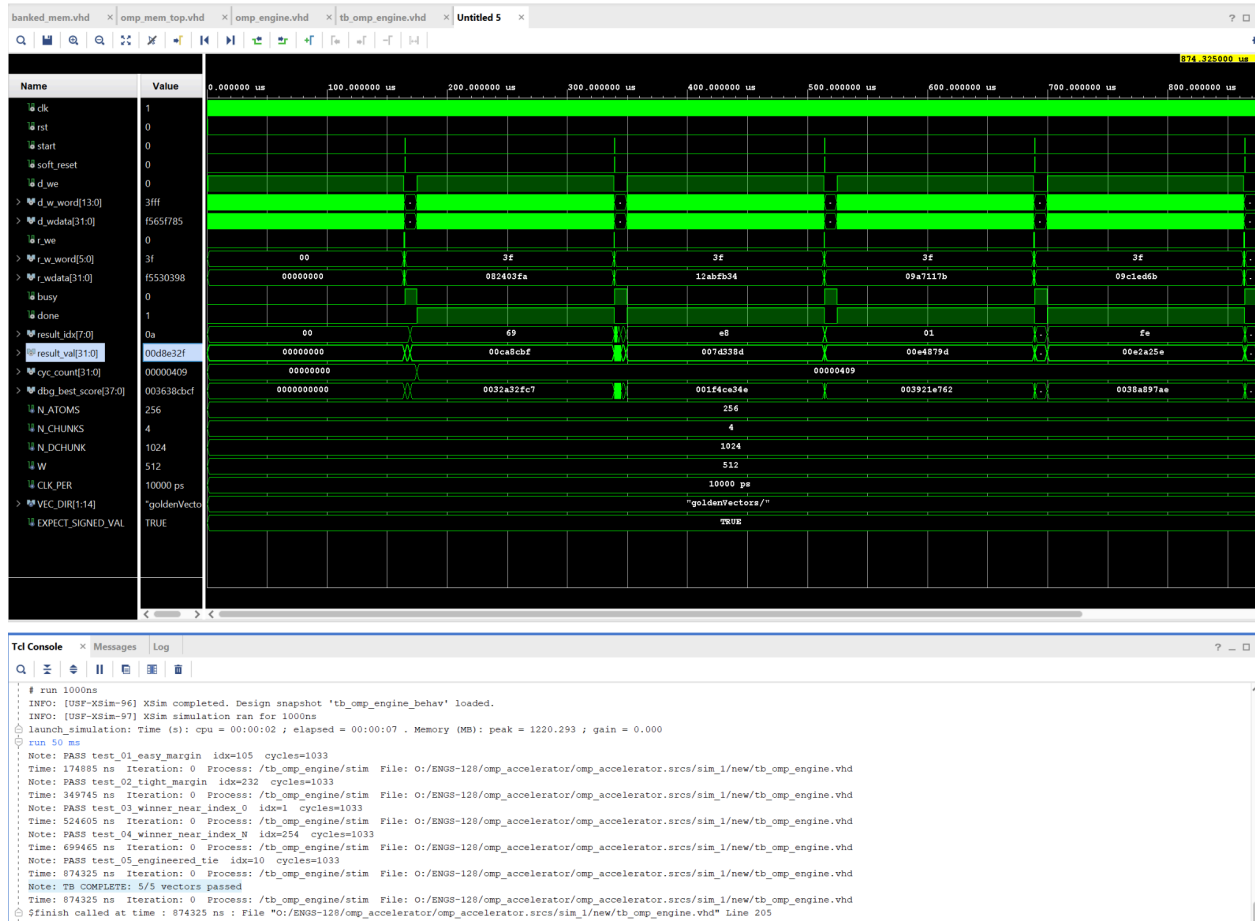
Vivado Generated Elaborated RTL Design for Top Level FSM

Design Task 1 Simulations

To test the full Design Task 1 pipeline, I had to make a ground truth source of vectors that we could compare to. This is where the software side of this project came into play. In Python, I created the full OMP recovery engine using the help of Claude AI. This gave me golden vectors that I can compare against when I run the recovery engine in hardware. Below are the 5 golden vectors that I tested, and as you can see at the bottom the test passes and all 5 are correctly obtained. This validated that the inner workings of the OMP engine were working properly. The five golden vectors were tested across a variety of circumstances:

1. Easy Margin: A clear winning vector with a large gap to second place. This test is meant to establish that we are recovering signals at all.
2. Tight Margin: A not-so-clear winner with a margin of $\sim 0.002\%$. This test is meant to test the accuracy of the engine.
3. Winner near last index: This is meant to test that the OMP engine was correctly timed and that we weren't dropping a correlation at the end.

- Winner near first index: Again meant to test that the OMP engine was correctly timed, but at the other end.
- Engineered tie: Two vectors have the same score, and according to theory we should grab the lowest index. This is meant to test how close to theory we applied our engine.



OMP Engine Simulation showing 5 Golden Vectors

Task 2: AXI4 Lite Wrapper

Configuration and Memory Component

The AXI4 Lite exposes a memory-mapped interface at the base address assigned in the Vivado block design (shown below). There are two distinct address spaces: the control register and the BRAM storage. Four control registers (CTRL, STATUS, RESULT_IDX, RESULT_VAL) are controlled by the processor to send commands to the programmable logic (PL). The CTRL register gets cleared by hardware after one clock cycle: this creates the START pulse used above.

STATUS is read only and reflects the engine's current state. RESULT_IDX holds the argmax output index, and is only valid when DONE (bit 0 of the STATUS register) is high.

We store 64kB of dictionary data and 256B of residual data on the FPGA for computation. The dictionary data is kept for the entire computation of recovery, but the residual data is updated once for each of the 32 correlation engine calls. The BRAM region is mapped into BRAM_D for the 128x256 dictionary, and BRAM_R for the 128-entry residual vector. Each vector is 2 bytes, thus it holds 256 bytes of data. The residual and dictionary data was kept in separate BRAMs so as to not introduce lagging issues when rewriting the residual per OMP loop. The dictionary is stored in atom-major where all 64 words for a specific atom occupy contiguous address space, which reduces latency and operation count while reading.

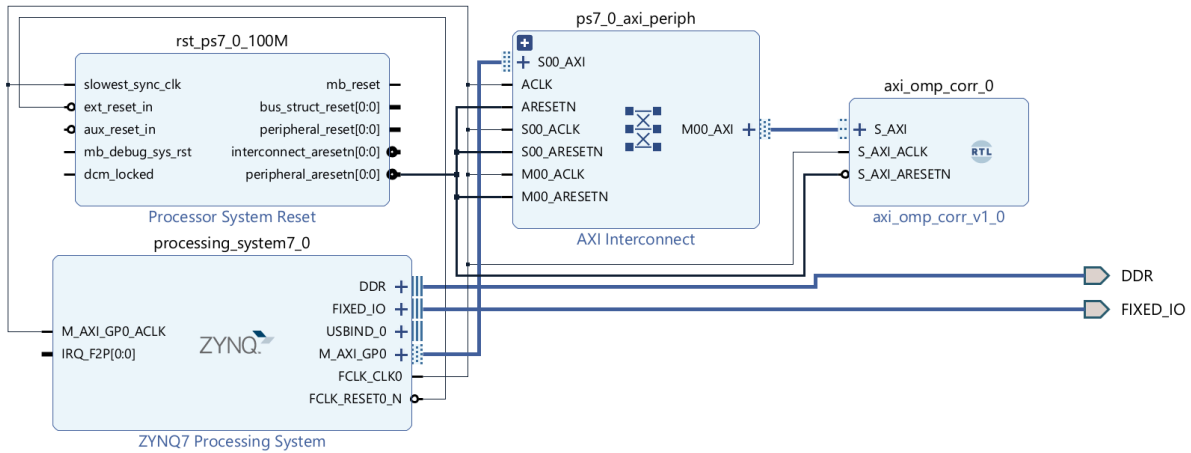
The memory was code generated using a coding assistant, as time on this project was running out and the rest of the system had to get connected together. The BRAM files are generic memory files that instantiate large arrays that hold the r and D dictionaries. The AXI4 Lite was grabbed from a template found online, and I routed my components and wires through the system. A coding assistant helped me with bugs and other weird issues.

The Vivado Generated block design is below, and it shows how Vivado split up our total BRAM allocation requirement across multiple components and blocks, something I find fascinating.



Full System Integration Component

The complete full system integration was assembled in Vivado's block design, which connects up the four major components: the Zynq processing system, the correlation engine IP, the AXI Interconnect, and the processing system reset infrastructure. The whole thing is wired up using Vivado's connection automation, which automatically routes the components. The PS is configured to output a 100 MHz fabric clock, which feeds into everything else. No external BRAM controller is needed as the BRAM is internal to my IP and is accessed directly using the AXI register map.



Vivado Block Design for the Full System Integration

Task 3: OMP Outer Loop (Bare-metal C)

The following design task deals with the OMP outer loop, and all components below are implemented in the Vitis IDE in C. Since these are logical components that are generic, there are no paper designs, schematics, or simulations. Their behavior was validated by running the code and observing the results. The entire pipeline was validated against a Claude generated Python recovery engine that provided golden vectors and other standard values and outputs to compare against. All of these Python files are found within the GitHub repository, with instructions on how to run them.

AXI Hardware Driver Component

The AXI driver is a thin C abstraction over memory-mapped I/O that wraps all interaction with the correlation engine control register. It provides three core operations to the OMP outer loop: load the dictionary D into BRAM_D once on startup, write the current residual dictionary r into BRAM_R each OMP iteration, and grab the argmax that was computed at the end of the 32

iterations. Register accesses use 32-bit pointer writes to the base address offset as defined in the *xparameters.h* file, which was created by Vivado when we exported the hardware file (.xsa) to Vitis. That transfer of hardware files is what ensures the addresses match up, and it's what allows us to program PS logic alongside PL logic. The BRAM packing, which consists of two Q1.15 values, is handled in the load function.

OMP Iteration and Least-Squares Component

The OMP outer loop runs in C entirely on the Cortex-A9 in bare-metal. All of the working arrays are statically sized to the maximum sparsity of 32 atoms, such that we don't need to rely on dangerous dynamic memory allocation problems. Below is how the data flows through the system, from first bootup until it we are provided the recovered signal coefficients back on the Cortex-A9:

1. The dictionary D is written to BRAM_D
2. The current residual r , comprising 128 Q1.15 values, is written to BRAM_R
3. The hardware runs the correlation engine, producing the *best_score* and *best_idx*
4. Index *best_idx* is appended to the support set
5. The sub-dictionary D_s is formed by the selected columns
6. The least-square coefficients are solved using normal equations
7. The residual is updated
8. Repeat steps 2-7 for 32 iterations
9. The y vector was originally down-scaled for overflow protection, thus we apply a post processing scaling by dividing our recovered coefficients by $\alpha = 0.9 / \max|y|$ to get back to our original scale.

We now have obtained the scaled, recovered coefficients for the signal at hand.

Inverse Wavelet Transform Component

Now that we have the recovered coefficients, we map the sparse vector back to the time domain via a dot product against the Psi vector. This Psi vector is the 256x256 Daubechies db4 wavelet synthesis matrix, and was chosen as the transformation matrix because the QRS complex is better represented in this wavelet domain. This choice does not affect the PL logic or layer at all, and is simply a design choice to transform the signal in the most applicable domain. Either way, the fabric is performing inner products of atoms, and produces the same results even when the transformation matrix changes.

Task 4: End-to-End Heartbeat Reconstruction Pipeline

UDP Streaming Component

Printing raw reconstruction results over UART didn't really portray how accurately the reconstruction engine produced the recovered signals, so instead, firmware streams each OMP iteration to a host machine over UDP using the lwIP RAW API running bare-metal on the Cortex-A9. The board holds a static IP of *192.168.1.10* and transmits 1032-byte frames to the Mac (the host) with an IP of *192.168.1.50* through port 5005 using a direct Ethernet cable. Each frame carries a header that includes window ID, iteration index, total iterations, and a flags byte, as well as the 256-sample reconstruction and 256-sample original signals as Q1.15 arrays. It also sends back the reconstruction error that is portrayed as Percent Root-mean-square Difference (PRD). The final design choice to stream every iteration rather than just the final result provided a beautiful animation of how the algorithm recovers the signal. Very cool stuff. The RAW API was created through a Vitis template, and adapted towards a static IP protocol. During bringup of the UDP, the boilerplate *xemacpsif_physpeed.c* needed a change that got rid of the autonegotiation between the PHY and the Zynq MAC, and 100 Mbps full-duplex agreement was enacted. After this fix was enacted, data was smoothly sent across the system.

Testing and Validation of Streaming Component

To test and validate the UDP Streaming Component, 4 tests were performed to make sure that the simulation validated engine performed the *exact* same in the real world environment. In fact, this testing part of the lab found a bug so deep in the OMP engine that it would have laid dormant until the exact edge case was reproduced in real time. Here we run the same 5 tests that we did above (testing the simple case all the way to edge cases), and low and behold, the "Winner near last index" performed incorrectly in the real world hardware test. It was in fact, an off-by-one error that I have since fixed. Below are all of the tests and their outputs. These tests were performed in Vitis using static software generated memory files that were then fed into the BRAMs using AXI4 Lite control registers and protocols. Then we would launch hardware in the Vitis terminal and listen on the correct port for the UART output. Below you can see the 5 tests (same as the 5 above) passing.

Connected to COM7 at 115200

```
=== OMP correlation engine — Phase 3 board bring-up ===  
STATUS = 0x00000010 (VERSION=0x1, DONE=0, BUSY=0)  
Loading D (256 atoms x 128) and r (128) into BRAM...  
RESULT_IDX = 137 (RESULT_VAL = 442173716, debug)  
expected = 137  
>>> PASS: on-board engine matches simulation. Hardware is alive.
```

Vitis Serial Terminal Real-Hardware OMP Test 1

```
=== OMP correlation engine — Phase 3 board bring-up ===  
STATUS = 0x00000010 (VERSION=0x1, DONE=0, BUSY=0)  
Loading D (256 atoms x 128) and r (128) into BRAM...  
RESULT_IDX = 100 (RESULT_VAL = 322329199, debug)  
expected = 100  
>>> PASS: on-board engine matches simulation. Hardware is alive.
```

Vitis Serial Terminal Real-Hardware OMP Test 2

```
=== OMP correlation engine — Phase 3 board bring-up ===  
STATUS = 0x00000010 (VERSION=0x1, DONE=0, BUSY=0)  
Loading D (256 atoms x 128) and r (128) into BRAM...  
RESULT_IDX = 0 (RESULT_VAL = 298578283, debug)  
expected = 0  
>>> PASS: on-board engine matches simulation. Hardware is alive.
```

Vitis Serial Terminal Real-Hardware OMP Test 3

```
=== OMP correlation engine — Phase 3 board bring-up ==  
STATUS = 0x00000010 (VERSION=0x1, DONE=0, BUSY=0)  
Loading D (256 atoms x 128) and r (128) into BRAM...  
RESULT_IDX = 69 (RESULT_VAL = 23536570, debug)  
expected = 255  
>>> FAIL: idx 69 != expected 255.
```

```
=== OMP correlation engine — Phase 3 board bring-up ===  
STATUS = 0x00000010 (VERSION=0x1, DONE=0, BUSY=0)  
Loading D (256 atoms x 128) and r (128) into BRAM...  
RESULT_IDX = 255 (RESULT_VAL = 325554900, debug)  
expected = 255  
>>> PASS: on-board engine matches simulation. Hardware is alive.
```

```

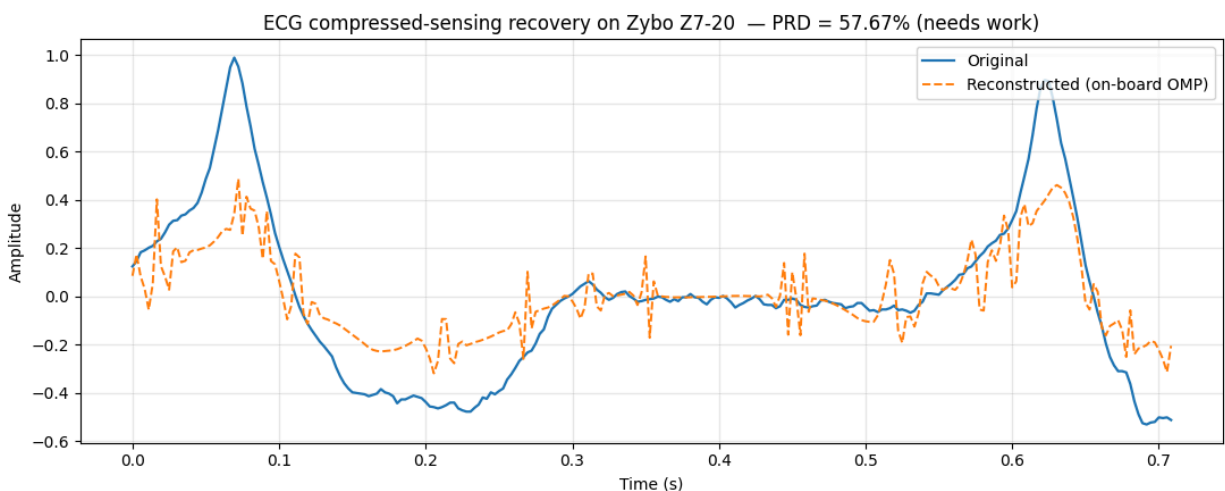
=== OMP correlation engine — Phase 3 board bring-up ===
STATUS = 0x00000010 (VERSION=0x1, DONE=0, BUSY=0)
Loading D (256 atoms x 128) and r (128) into BRAM...
RESULT_IDX = 50 (RESULT_VAL = 265560582, debug)
expected   = 50
>>> PASS: on-board engine matches simulation. Hardware is alive.

```

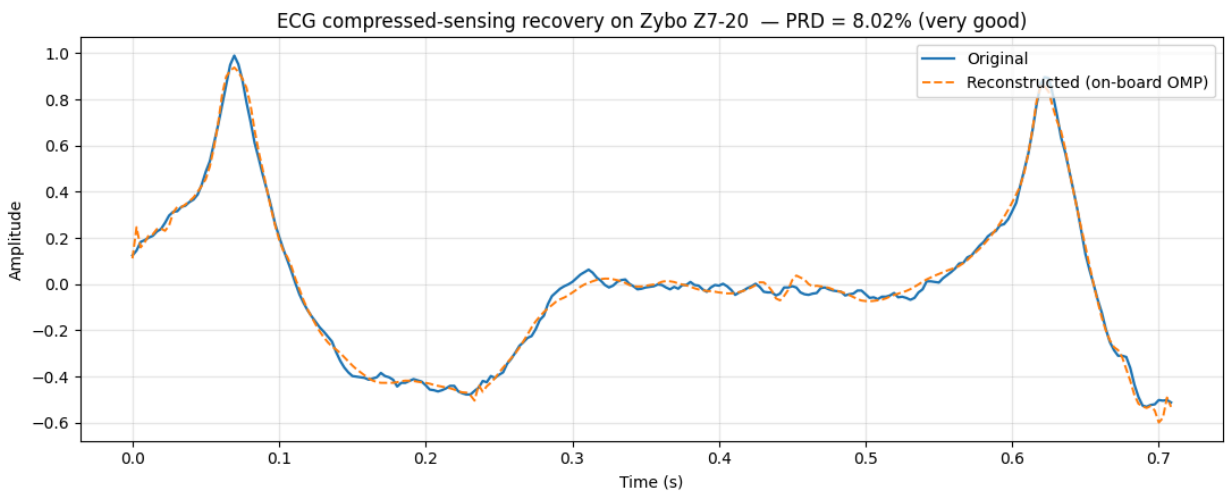
Visualization Component

On the receiving side of all of this data was a Python parser and visualization file *animate.py* that listened on a UDP socket and rendered incoming reconstruction frames as a live *matplotlib* animation. The animation plays back the 32 step reconstruction 400ms per frame (although this is essentially instantaneous in physical hardware, we implement a timer to make it visible), showing the original ECG signal overlaid with the current OMP reconstruction, providing the current PRD in the title of the plot. This produces a clean, slowly converging signal that you can see eventually mapped quite nicely onto the original signal. Multiple signals were tested through the pipeline, many of them handpicked for their level of sparsity.

Below I show you a progression of signal recovery, from the first really bad result to the clinically acceptable result. The main difference between the below images is actually what I referred to earlier, the transformation matrix used to reconstruct the recovered coefficients. Originally I was transforming using a DCT basis, but this basis does not do well in describing sharp spikes, like the one found in heart beats. That is why I swapped to the db4 wavelet basis. As you can see, the contrast is stunning.



First Recovery of the Sparse Heartbeat Signal



Second Recovery of the Sparse Heartbeat Signal

Post-Lab Discussion

Methodology

For each design task I followed the incremental build and test process which had me doing the following: Paper Design → VHDL Writeup → Elaboration Visual Check → Simulation Validation. I first started with the paper design, which forced me to understand where every data signal in the system went and how it affected each sub-component. This was extremely helpful because before I even touched Vivado I was able to have a clear picture of the component in my head.

The elaboration check gave me a visual description of where the data was flowing within my system. I always checked this against my paper design to 1) see if I implemented the logic of the system correctly, and 2) see where Vivado optimized out signals and processes. The second part was fascinating, with some optimizations taking me a while to understand the reasoning behind.

I addressed a lot of challenges that I faced throughout the above documentation, but the worst challenge occurred on presentation day. The night before I had the entire pipeline working. I would start up the Vitis hardware application manager, I would run the *animate.py* file on my Mac, and I would get reconstructed signals plotted nicely on an animation in *matplotlib*. This was fine for 4 hours the night before, until 2am. Then, I wake up, have breakfast, go to the exact same lab, sit down and change literally nothing, and the system doesn't want to accept data. I

debug for 6 total hours before the 3:30pm presentation, and I cannot get my Mac to receive data. Luckily I had a video of it working from the night before, but I could not perform a live demo and that frustrated me. The problem was that my UDP connection on the Mac side would just refuse to secure the incoming data correctly (maybe it would drop it??), and I never got another plot again. I tried for multiple hours the next day trying to figure out why I couldn't receive reconstruction data, but alas nothing worked. Right now, I think it's an IP and firewall problem (this is data coming from my Dad, who has worked in IT for his whole life). I think that the static IP system that I set up is biting me, and that maybe getting the autonegotiation to work would have been the move. Either way, you can stream data back to the monitor the FPGA is hooked up to through UART, and still obtain the data needed for plotting. In light of this UDP function, I have another Python script that converts the raw data from a text file into the plot of the reconstructed image.

Out of all of the designs from labs 1-3, I didn't use any of the components as I dropped the audio pipeline completely. None of the components were cleanly mapped across the very separate and unique domains that I solved this term, thus this project was started from scratch.

If I had a do-over for this term, I would figure out how to get the UDP working cleanly and efficiently. I would also look at better compressed sensing techniques, as I read about a possibility of doing the same heartbeat signal-recovery flow but with 75% less measurements (64 instead of 256) and quantizing down to 1-bit instead of 16 bits. Yea you heard me right, 1 single bit. Compressed Sensing has provided a theory that proves I can reconstruct a heart beat in 64 bits. That is the length of a modern CPU's address pointer (if you have a beefy CPU). Incredible, but that's what I would shoot for.

Results

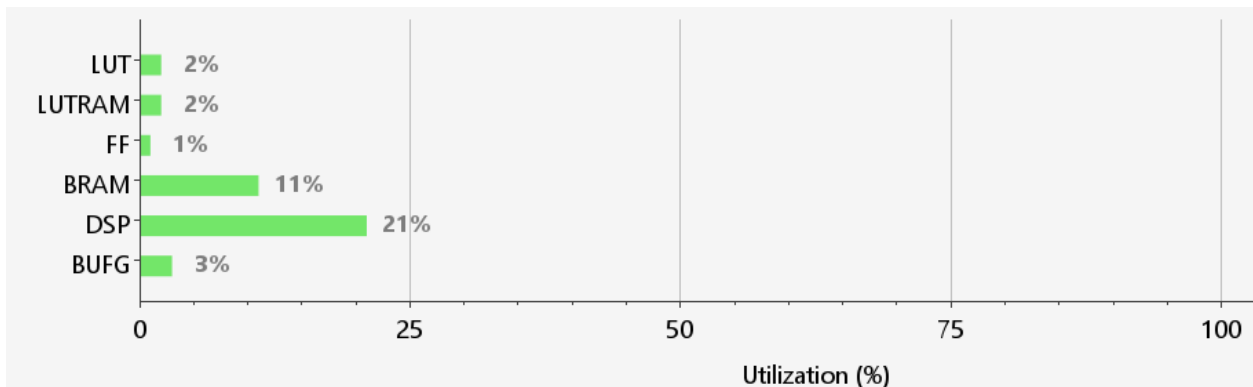
The system worked perfectly as intended and expected, until the live demo part, which is still such an annoying problem. When I tell you I literally changed nothing, I literally didn't even open my laptop from the moment I closed it at the lab station to the moment I got back and opened it up. Other than this weird ethernet bug, the system again worked as intended. Since most of this project was a very spelled out mathematical implementation, I never really observed crazy unexpected behavior or timing discrepancies, and was quite lucky (I made it this way so I guess it's not really "luck") that my whole system ran on the same clock.

My implemented design meets all timing constraints from the target 100 MHz clock, with all total delay under the 10 ns period. The worst negative slack, and thus the critical path, has a slack of 1.131 ns , running from the correlation engine's address-counter register into a multiplier input. The path is dominated by the *Net Delay*, indicating that it's a routing slack and not a logical bottleneck. This is consistent with how the design has been implemented – 32 parallel MACs needing data all at once can cause some routing nightmares, thank god for automation.

Name	Slack	L...	...	^1	From	To	Total Delay	Logic Delay	Net Delay	Requirement
Path 1	1.131	1	132		omp_acc_bd1_i/axi_omp_corr...addr_cnt_reg[1]_rep_2/C	omp_acc_bd1_i/axi...e/A_reg[3]/B[16]	8.082	0.609	7.473	10.0
Path 2	1.131	1	132		omp_acc_bd1_i/axi_omp_corr...addr_cnt_reg[1]_rep_2/C	omp_acc_bd1_i/axi...e/A_reg[3]/B[17]	8.082	0.609	7.473	10.0
Path 3	1.170	1	132		omp_acc_bd1_i/axi_omp_corr...addr_cnt_reg[1]_rep_0/C	omp_acc_bd1_i/axi...u_mac/p_reg/B[2]	8.070	0.670	7.400	10.0
Path 4	1.203	0	736		omp_acc_bd1_i/axi_omp_corr_0/U0/r_w_word_reg[5]/C	omp_acc_bd1_i/...17/RAMA/WADR1	8.629	0.456	8.173	10.0
Path 5	1.203	0	736		omp_acc_bd1_i/axi_omp_corr_0/U0/r_w_word_reg[5]/C	omp_acc_bd1_i/...RAMA_D1/WADR1	8.629	0.456	8.173	10.0
Path 6	1.203	0	736		omp_acc_bd1_i/axi_omp_corr_0/U0/r_w_word_reg[5]/C	omp_acc_bd1_i/...17/RAMB/WADR1	8.629	0.456	8.173	10.0
Path 7	1.203	0	736		omp_acc_bd1_i/axi_omp_corr_0/U0/r_w_word_reg[5]/C	omp_acc_bd1_i/...RAMB_D1/WADR1	8.629	0.456	8.173	10.0
Path 8	1.203	0	736		omp_acc_bd1_i/axi_omp_corr_0/U0/r_w_word_reg[5]/C	omp_acc_bd1_i/...17/RAMC/WADR1	8.629	0.456	8.173	10.0
Path 9	1.203	0	736		omp_acc_bd1_i/axi_omp_corr_0/U0/r_w_word_reg[5]/C	omp_acc_bd1_i/...RAMC_D1/WADR1	8.629	0.456	8.173	10.0
Path 10	1.203	0	736		omp_acc_bd1_i/axi_omp_corr_0/U0/r_w_word_reg[5]/C	omp_acc_bd1_i/a...2_17/RAMD/ADR1	8.629	0.456	8.173	10.0

Timing Report Showing We Meet Specifications

As you can see below, I meet the resource requirements needed for the FPGA on hand. My design is not fairly complex in nature, and so we don't use that many resources. This is a good thing, and should be one of the main points taken away from this project: 2% LUT, 1% FF, 11% BRAM, and 21% DSP, alongside the fact that we are recovering heartsignals, gives us either room to make our device smarter and/or more powerful, or small enough that it can fit within the embedded human device.



Resource Allocation Graph Showing Low Consumption

The implemented design compiles with zero errors. The ~220 messages are benign warnings generated by other, non-core project components. The 62 implementation warnings are all about a single DRC class, and it suggests that pipelining will improve timing constraints. This is not needed as we already showed above that we meet timing, but if we ever needed to get more wiggle room, pipelining is the way to go.

```

  - [a Synthesis (157 warnings)
    - [a synth_1 (11 warnings)
      - [a [Synth 8-7129] Port M_AXI_CLK in module x00_couplers_imp_1HVMURU is either unconnected or has no load (7 more like this)
      - [a [Designutils 20-4072] Reference run did not run incremental synthesis because the design is too small: reverting to default synthesis (1 more like this)
      - [a [Synth 8-7080] Parallel synthesis criteria is not met
    - [a Out-of-Context Module Runs (146 warnings)
      - [a omp_acc_bd1 (146 warnings)
        - [a omp_acc_bd1_processing_system7_0_0_synth_1 (115 warnings)
          - [a [Synth 8-7071] port 'M_AXI_GP0_ARESETN' of module 'processing_system7_v5_5_processing_system7' is unconnected for instance 'inst' [omp_acc_bd1_processing_system7_0_0_v327] (12 more like this)
          - [a [Synth 8-7023] Instance 'inst' of module 'processing_system7_v5_5_processing_system7' has 685 connections declared, but only 672 given [omp_acc_bd1_processing_system7_0_0_v327]
          - [a [Synth 8-7129] Port ENET0_GMII_COL in module processing_system7_v5_5_processing_system7 is either unconnected or has no load (99 more like this)
          - [a [Synth 8-7080] Parallel synthesis criteria is not met
        - [a omp_acc_bd1_rst_ps7_0_100M_0_synth_1 (10 warnings)
          - [a [Synth 8-7129] Port prmyr_adc in module cdc_sync is either unconnected or has no load (6 more like this)
          - [a [Synth 8-7080] Parallel synthesis criteria is not met
          - [a [Synth 8-3332] Sequential element (EXT_LPF/ACTIVE_LOW_EXT_ACT_IQ_EXT_GENERATE_LEVEL_P_5_CDC_SINGLE_BIT_CROSS_PLEVEL_INQ2SNDRY_s_level_out_d5) is unused and will be removed from module proc_sys_reset. (3 more like this)
        - [a omp_acc_bd1_axi_omp_corr_0_0_synth_1 (21 warnings)
          - [a [Synth 8-6014] Unused sequential element score_abs_reg was removed. [regmap_tracker.vhd:79] (1 more like this)
          - [a [Synth 8-7129] Port S_AXI_AWPROT[2] in module axi_omp_corr is either unconnected or has no load (17 more like this)
          - [a [Synth 8-7080] Parallel synthesis criteria is not met
    - [a Implementation (62 warnings)
      - [a Write Bitstream (62 warnings)
        - [a DRC (62 warnings)
          - [a Netlist (62 warnings)
            - [a Instance (62 warnings)
              - [a Pipeline (62 warnings)
                - [a DSP48E1 (62 warnings)
                  - [a [DRC DPRP-1] Input pipelining: DSP omp_acc_bd1_axi_omp_corr_0_0_0/u_mem_top/u_engine/u_dp/gen_mac[0]u_mac/p_req input omp_acc_bd1_axi_omp_corr_0_0_0/u_mem_top/u_engine/u_dp/gen_mac[0]u_mac/p_req/A2940 is not pipelined. Pipelining DSP48 input will improve performance. (61 more like this)

```

Vivado Messages and Warnings Printout showing Benign Warnings

Reflection

The coolest thing I learned in this course was how to make a tiny chip perform calculations blazingly fast. It opened my eyes into the world of fast computation, and at the end with this final project, into the world of embedded medical devices. I am very excited to take my FPGA skills into the real world and produce some life changing engineering products. I learned how to use complex system software such as Vivado and Vitis to leverage my preexisting computer science foundation to produce truly powerful systems, and I can't wait to dive into the world of personal projects within FPGAs.

I wish I understood how to make my processes even faster, how to use Vivado to the fullest intent, and how to parallelize anything and everything. I learned through my personal project how to instantiate multiple components (such as the MAC units), in order to perform high throughput data movement, and although I learned a lot about IP cores and the base Vivado functions, I believe that parallel compute could have been talked about more.

The most exciting part of my system is the potential to help people. When I started looking into the real world applications of signal recovery and embedded FPGA systems, I became very motivated to get this project done. I would love to take this project and push it even further, pushing the compute up to the limits of the FPGA. Using compressed sensing and FPGA knowledge, I can recover biomedical signals more efficiently, providing health monitors longer battery life and patients more comfort. This is the type of engineering that drives and motivates me, and I thank you (yes you! The reader Professor Kendall) for providing me with an amazing class and learning opportunity this term.

Below is a *gif* showing what the ethernet connection would have shown if it didn't decide to ruin my Wednesday afternoon.

